

Software Engineering Mathematics

Anonymised Submission ^{*†}

October 2025

^{*}Software Engineering Programme
[†]University of Oxford

Contents

Question 1	3
Question 2	3
Question 3	4
Question 4	7
Question 5	7
Question 6	8
Question 7	9
Question 8	10
Question 9	11
Question 10	12
Question 11	12
Question 12	13

Question 1

A tautology is a statement that is true for every valuation of the propositional variables it contains. A contradiction is a statement that is false for each such valuation. A contingency is a statement that is neither a tautology nor a contradiction. [?, Page 13].

(a)

p	$\neg p$	$(\neg p \Rightarrow p)$	$\mathbf{p} \Rightarrow (\neg \mathbf{p} \Rightarrow \mathbf{p})$
t	f	t	$\mathbf{t}$
f	t	f	$\mathbf{t}$

$p \Rightarrow (\neg p \Rightarrow p)$ is a tautology as it is true for every interpretation of p as illustrated by the final column of truth values.

(b)

p	q	$(q \Rightarrow p)$	$\neg p$	$\neg q$	$(\neg p \Rightarrow \neg q)$	$(\neg \mathbf{p} \Rightarrow \neg \mathbf{q}) \Leftrightarrow (\mathbf{q} \Rightarrow \mathbf{p})$
t	t	t	f	f	t	$\mathbf{t}$
t	f	t	f	t	t	$\mathbf{t}$
f	t	f	t	f	f	$\mathbf{t}$
f	f	t	t	t	t	$\mathbf{t}$

The statements $(\neg p \Rightarrow \neg q)$, $(q \Rightarrow p)$ are equivalent and $(\neg p \Rightarrow \neg q) \Leftrightarrow (q \Rightarrow p)$ is a tautology as it is true for every interpretation of p , q as illustrated by the final column of truth values.

(c)

p	q	r	$(p \Rightarrow r)$	$(q \Rightarrow r)$	$((p \Rightarrow r) \wedge (q \Rightarrow r))$	$(p \vee q)$	$((p \vee q) \Rightarrow r)$	$((\mathbf{p} \Rightarrow \mathbf{r}) \wedge (\mathbf{q} \Rightarrow \mathbf{r})) \Leftrightarrow ((\mathbf{p} \vee \mathbf{q}) \Rightarrow \mathbf{r})$
t	t	t	t	t	t	t	t	$\mathbf{t}$
t	t	f	f	f	f	t	f	$\mathbf{t}$
t	f	t	t	t	t	t	t	$\mathbf{t}$
t	f	f	f	t	f	t	f	$\mathbf{t}$
f	t	t	t	t	t	t	t	$\mathbf{t}$
f	t	f	t	f	f	t	f	$\mathbf{t}$
f	f	t	t	t	t	f	t	$\mathbf{t}$
f	f	f	t	t	t	f	t	$\mathbf{t}$

The statements $((p \Rightarrow r) \wedge (q \Rightarrow r))$, $((p \vee q) \Rightarrow r)$ are equivalent and $((p \Rightarrow r) \wedge (q \Rightarrow r)) \Leftrightarrow ((p \vee q) \Rightarrow r)$ is a tautology as it is true for every interpretation of p , q , r as illustrated by the final column of truth values.

Question 2

(a)

$$\begin{aligned}
 & p \Rightarrow (\neg p \Rightarrow p) \\
 & \Leftrightarrow p \Rightarrow (\neg \neg p \vee p) && [\Rightarrow \text{disjunction}] \\
 & \Leftrightarrow p \Rightarrow (p \vee p) && [\text{negation is an involution}] \\
 & \Leftrightarrow p \Rightarrow p && [\text{idempotence of } p] \\
 & \Leftrightarrow \text{true} && [\text{tautology}]
 \end{aligned}$$

In this equivalence, we prove that $p \Rightarrow (\neg p \Rightarrow p)$ by starting with implication disjunction to replace the implication with a logical or [?, page 17]. Next, we use the definition of double negation [?, Slide 25] and the application of the idempotence rule to simplify [?, Slide 26]. We trivially conclude with the fact that $p \Rightarrow p$ is a tautology.

(b)

$$\begin{aligned}
& \neg p \Rightarrow \neg q \\
& \Leftrightarrow \neg \neg p \vee \neg q & [\Rightarrow \text{disjunction}] \\
& \Leftrightarrow p \vee \neg q & [\text{negation is an involution}] \\
& \Leftrightarrow \neg q \vee p & [\text{commutativity of } \vee] \\
& \Leftrightarrow q \Rightarrow p & [\Rightarrow \text{disjunction}]
\end{aligned}$$

In this equivalence proof, we show that $(\neg p \Rightarrow \neg q) \Leftrightarrow q \Rightarrow p$ by using implication disjunction to replace implication with logical or, the definition of double negation to simplify, the commutivity property of logical or to change the order of the elements [?, Slide 27], and the implication disjunction rule to reintroduce implication.

(c)

$$\begin{aligned}
& (p \Rightarrow r) \wedge (q \Rightarrow r) \\
& \Leftrightarrow (\neg p \vee r) \wedge (\neg q \vee r) & [\Rightarrow \text{disjunction}] \\
& \Leftrightarrow (r \vee \neg p) \wedge (r \vee \neg q) & [\text{commutativity of or}] \\
& \Leftrightarrow r \vee (\neg p \wedge \neg q) & [\text{distributivity 2}] \\
& \Leftrightarrow (\neg p \wedge \neg q) \vee r & [\text{commutativity of } \vee] \\
& \Leftrightarrow \neg (p \vee q) \vee r & [\text{De Morgan 1}] \\
& \Leftrightarrow p \vee q \Rightarrow r & [\Rightarrow \text{disjunction}]
\end{aligned}$$

In this equivalence proof, we show that $((p \Rightarrow r) \wedge (q \Rightarrow r)) \Leftrightarrow (p \vee q) \Rightarrow r$ by using the implication disjunction rule to replace implication with logical $\vee$, the commutativity of $\vee$ rule to change the order of the elements [?, Slide 27], the distributivity 2 rule to extract r [?, Slide 29], and the commutivity of logical $\vee$ to change the order of the elements, De Morgan's Law to extract negation for $\neg (p \text{ and } \neg q)$ [?, Slide 28], and the implication disjunction rule to reintroduce implication.

Question 3

(a)

$$\frac{\frac{\frac{\frac{\Gamma p \neg[1] \quad \Gamma \neg p \neg[3]}{\text{false}} [\text{false-intro}]}{\text{false}} [\text{false-elim}^{[3]}]}{\frac{\Gamma \neg p \neg[2]}{p}} [\neg\text{-intro}^{[2]}]}{\frac{\Gamma p \neg[1]}{p \Rightarrow (\neg p \Rightarrow p)}} [\Rightarrow\text{-intro}^{[1]}]$$

For this proof we are seeking to prove that $p \Rightarrow (\neg p \Rightarrow p)$, which we know holds given the truth table in 1(a). Working mechanically from the bottom, we start by applying the implication introduction rule to identify and discharge assumption [1] to gain our conclusion [?, Slide 3] and again we use the implication introduction rule to identify and discharge assumption [2] to gain $(\neg p \Rightarrow p)$. We note that this statement requires that we have both p and $\neg p$ as premisses, which implies we need to use false in the proof. With some experimentation, we realize we need three assumptions and we introduce a third assumption [3]. We first use false-intro followed by false-elim1 to discharge assumption [3] [?, Slide 6]. In the final proof, all assumptions are discharged properly. We note that this proof bears some similarities with the rules of negation proof in the course notes section 4.8 [?, Pages 41-46] which has multiple degrees of negation of p as assumptions. The key is to have $\neg p$ as an assumption twice at different scopes.

(b)

This proof involves equivalence or bi-implication.

In one direction, we are seeking to prove $(\neg p \Rightarrow \neg q) \Rightarrow (q \Rightarrow p)$:

$$\frac{\frac{\frac{\frac{\neg p \Rightarrow \neg q^{[1]} \quad \neg p^{[3]}}{\neg q} [\Rightarrow \text{elim}] \quad \neg q^{[2]}}{\text{false}} [\text{false-elim}^{[3]}] \quad p [\Rightarrow \text{-intro}^{[2]}]}{\neg p \Rightarrow \neg q^{[1]} \quad q \Rightarrow p} [\Rightarrow \text{-intro}^{[1]}]}{\neg p \Rightarrow \neg q \Rightarrow (q \Rightarrow p)}$$

I started working mechanically from the bottom by applying the implication introduction rule to construct the conclusion. This gave me $q \Rightarrow p$ as the key premise to build up, which requires applying the implication introduction rule to q and p . Looking at the need for q and looking at the need for p and $(\neg p \Rightarrow \neg q)$ as assumption [1], I felt it would be best to try not p as an assumption to at least get going. This enabled me to work top down to get not q from the implication elimination rule. From there, I realized that I needed the negation of both not q and not p . I knew the false-intro and false-elim were required, but it took some experimentation to see that I could get both $\neg q$ and not p transformed into p and q by making not p assumption [3] instead of [2] and introducing q as assumption [2]. Once that insight was made, I could arrive at p as a result of the top of the tree and retain q [2] as an assumption coming out of false-elim. This is what I needed to arrive at $q \Rightarrow p$ (and) to connect the bottom and top portions of my proof. All assumptions are properly discharged.

and in the other, we are seeking to prove $(q \Rightarrow p) \Rightarrow (\neg p \Rightarrow \neg q)$:

$$\frac{\frac{\frac{\frac{q \Rightarrow p^{[1]} \quad q^{[3]}}{p} [\Rightarrow \text{elim}] \quad \neg p^{[2]}}{\text{false}} [\text{false-elim}^{[3]}] \quad \neg q [\Rightarrow \text{-intro}^{[2]}]}{\neg p^{[1]} \quad \neg p \Rightarrow \neg q} [\Rightarrow \text{-intro}^{[1]}]}{\neg p \Rightarrow \neg q \Rightarrow (q \Rightarrow p)}$$

Working mechanically from the bottom, I apply the implication rule to $q \Rightarrow p$ as assumption [1] to gain the conclusion. I need $(\neg p \Rightarrow \neg q)$, which shows me to use the implication rule again with $\neg p$ as assumption [2]. From there, the question is how to get $\neg q$.

Now working from the top, given assumption [1] has q , it makes sense to use the implication elimination rule with q as assumption [3]. This gives me p , when what I want is $\neg q$. I see that if I introduce $\neg p$ as assumption [3] that I can negate q via the false-intro and false-elim rules and successfully discharge assumption [3]. This gives me $\neg q$, which is what I need to connect to the top tree to the bottom of the proof. This proof came more quickly because $q \Rightarrow p$ as assumption [1] gave clear direction for the top portion. All assumptions are discharged and scopes are appropriate.

We then combine these two proofs with the $\Leftrightarrow$ -intro rule [?, Slide 4], using the results from above to conclude our proof.

$$\frac{(\neg p \Rightarrow \neg q) \Rightarrow (q \Rightarrow p) \quad (q \Rightarrow p) \Rightarrow (\neg p \Rightarrow \neg q)}{(\neg p \Rightarrow \neg q) \Leftrightarrow (q \Rightarrow p)} [\Leftrightarrow \text{intro}]$$

In one direction, we are seeking to prove $((p \Rightarrow r) \wedge (q \Rightarrow r)) \Rightarrow ((p \vee q) \Rightarrow r)$

In this proof, I start by working from the bottom mechanically, taking $((p \Rightarrow r))(and(q \Rightarrow r))$ as assumption [1] and applying the implication introduction rule to arrive at our conclusion and discharge assumption [1]. From there, I can see that I need $(p \vee q)$ and r as inputs to the implication introduction rule. I take $(p \vee q)$ as assumption [2] and discharge this assumption via the implication introduction rule. That leaves me with r as the result of my bottom up process.

At this stage, I do not discharge assumption [2], as this is deferred until the next step. I use implication introduction together with (p or q) [2] and r from the or elimination to discharge assumption [2], which connects the top-down and bottom-up process. All assumptions are properly discharged.

[illegible]

Working mechanically from the bottom, we start by taking $((p \vee q) \Rightarrow r)$ as assumption [1] $\wedge$ applying the implication introduction rule to discharge assumption [1]. This leaves us with $((p \Rightarrow r) \wedge (q \Rightarrow r))$, which is derived from $p \Rightarrow r, q \Rightarrow r \wedge$ the application of the $\wedge$ introduction rule. Fairly clearly, we need two sub-branches to be developed top-down. Since we have $((p \vee q) \Rightarrow r) \wedge$ we want $p \Rightarrow r \wedge q \Rightarrow r$, I start with p [2] $\wedge$ q [3] to start the top-down sub-trees. I struggled here until I recalled that I could use the $\vee$ intro rule to satisfy $(p \vee q)$. I also had to recall that this does not generate additional assumptions to discharge [?, slide 38]. From here,

it was not difficult to use implication elimination to arrive at r in both sub-trees $\wedge$ to apply p [2] and q [3] in each tree with the implication introduction rule to discharge those assumptions. This gave me $p \Rightarrow r$, $q \Rightarrow r$ and I used the and introduction rule to connect the top and bottom portions of the proof. All assumptions are properly discharged.

We then combine these two proofs with the $\Leftrightarrow$ -intro rule, using the results from above to conclude our proof:

$$\frac{((p \Rightarrow r) \wedge (q \Rightarrow r)) \Rightarrow ((p \vee q) \Rightarrow r) \quad ((p \vee q) \Rightarrow r) \Rightarrow ((p \Rightarrow r) \wedge (q \Rightarrow r))}{((p \Rightarrow r) \wedge (q \Rightarrow r)) \Leftrightarrow ((p \vee q) \Rightarrow r)} [\Leftrightarrow \text{intro}]$$

Question 4

We are defining a generic function that will return the difference in length between the longest and shortest sequences from an input type that is a non-empty sequences of sequences. The input could be $\langle\langle x, y, z \rangle, \langle \rangle\rangle$ yielding 3 as output or $\langle\langle x \rangle\rangle$ yielding 0, for example. I specify three generic *functions* : *maxSeq* yields the longest sequence from a non-empty sequence of sequences; *minSeq* yields the shortest sequence from a non-empty sequence of sequences; and *maxDiff*, which fulfills the specification for Question 4.

$$\begin{array}{l} \boxed{[X]} \\ \hline \text{maxSeq} : \text{seq}_1(\text{seq } X) \rightarrow \text{seq } X \\ \text{minSeq} : \text{seq}_1(\text{seq } X) \rightarrow \text{seq } X \\ \text{maxDiff} : \text{seq}_1(\text{seq } X) \rightarrow \mathbb{N} \\ \hline \forall ss : \text{seq}_1(\text{seq } X) \bullet \text{maxSeq}(ss) = (\mu s : \text{ran } ss \mid \forall t : \text{ran } ss \bullet \#s \geq \#t) \\ \forall ss : \text{seq}_1(\text{seq } X) \bullet \text{minSeq}(ss) = (\mu s : \text{ran } ss \mid \forall t : \text{ran } ss \bullet \#s \leq \#t) \\ \forall ss : \text{seq}_1(\text{seq } X) \bullet \text{maxDiff}(ss) = (\#(\text{maxSeq}(ss)) - \#(\text{minSeq}(ss))) \end{array}$$

For *maxSeq* and *minSeq*, a μ function is defined that selects a single element meeting the specification based on comparing every sequence s with every sequence t . First, the range operator moves us from a sequence of sequences with ss values such as $\langle 1 \mapsto \langle x, y, z \rangle, 2 \mapsto \langle \rangle \rangle$ to sequences s, t with values like $\langle x, y, z \rangle, \langle \rangle$ to enable *seq* operators. *maxSeq* and *minSeq* use the operators and *maxDiff* uses the length operators with subtraction [?, Page 40]. For *maxDiff*, $\text{maxSeq} \geq \text{minDiff}$ is a valid pre-condition by virtue of *maxSeq* and *minSeq* and therefore the subtraction is guaranteed to return a natural number as defined in the specification, making *maxSeq()* a total function [?, Page 119]. This solution passes fuzz validation.

Question 5

The relational composition operator is associative:

$$(R \circ S) \circ T = R \circ (S \circ T) \\ \text{if } R \in W \leftrightarrow X, S \in X \leftrightarrow Y, \wedge T \in Y \leftrightarrow Z.$$

In this equivalence proof, we show that composition is associative using the definition of relational composition and by repositioning quantifiers and predicates where we are permitted to do so, which is only when there is no impact to the truth value of the predicates involved. Each such repositioning or regrouping will be justified.

Given a relation $R \in X \leftrightarrow Y$ and a second relation $S \in Y \leftrightarrow Z$, $\wedge$ two elements $x \in X \wedge z \in Z$, $x \mapsto z \in R \circ S \Leftrightarrow \exists y : Y \bullet x \mapsto y \in R \wedge y \mapsto z \in S$ [?, page 107].

$$\begin{aligned}
& w \mapsto z \in (R \circ S) \circ T \\
& \Leftrightarrow \exists y : Y \bullet w \mapsto y \in (R \circ S) \wedge y \mapsto z \in T & [\text{definition of } \circ] \\
& \Leftrightarrow \exists y : Y \bullet (\exists x : X \bullet w \mapsto x \in R \wedge x \mapsto y \in S) \wedge y \mapsto z \in T & [\text{definition of } \circ] \\
& \Leftrightarrow \exists y : Y; x : X \bullet w \mapsto x \in R \wedge x \mapsto y \in S \wedge y \mapsto z \in T & [x \text{ is not free in } y \mapsto z \in T] \\
& \Leftrightarrow \exists x : X \bullet w \mapsto x \in R \wedge (\exists y : Y \bullet x \mapsto y \in S \wedge y \mapsto z \in T) & [y \text{ is not free in } w \mapsto x \in R] \\
& \Leftrightarrow \exists x : X \bullet w \mapsto x \in R \wedge x \mapsto z \in (S \circ T) & [\text{definition of } \circ] \\
& \Leftrightarrow w \mapsto z \in R \circ (S \circ T)
\end{aligned}$$

In step 1, we apply the composition equivalence to the outer composition of $(R \circ S) \circ T$. In step 2, we apply our equivalence to the inner composition $(R \circ S)$. In step 3, we include the predicate $y \mapsto z \in T$ in the scope of $x : X$. We can do this because this does not affect the truth values of $y \mapsto z \in T$ as x is not free in this predicate. In step 4, we reposition the $\exists y : Y$ quantifier to be after the $w \mapsto x \in R$ predicate without affecting its truth values because y is not free in $w \mapsto x \in R$. In step 5, we apply the equivalence for composition to $S \circ T$. In step 6, we apply the equivalence for composition to $R \circ (S \circ T)$.

I worked on this proof both top-down and bottom-up. After doing the obvious equivalence substitutions, I was left with step 3 and 4 to work out. In working out step 3, I could see from step 6 that I wanted $S \circ T$ together. In working out step 4, I could see that $y : Y$ needed to be repositioned to setup step 6. The reasoning about both of these steps was not obvious at first, but I realized that these moves had no impact on the truth values of the affected predicates. This led me to think about free and bound variables and what rules might apply in various cases [?, Pages 54-55]. I was not able to find a specific justification in the materials that maps to the idea that one can reposition quantifiers or predicates provided there is no impact to the truth values, but this seems logical and sound provided x is not free in P for the affected P . I note the parallels with the logic for substitution operations [?, Pages 56-57].

Question 6

The following set comprehensions are presented using Z axiomatic definitions to enable direct fuzz validation of the LaTeX. Further, exponents are written long-hand as fuzz does not support exponentiation [?].

(a)

$$\begin{array}{|l}
\text{SquaresOfEvens} : \mathbb{P} \mathbb{Z} \\
\hline
\text{SquaresOfEvens} = \{ z : \mathbb{Z} \mid z \bmod 2 = 0 \bullet z * z \}
\end{array}$$

(b)

$$\begin{array}{|l}
\text{CubesOfSquaresOfEvens} : \mathbb{P} \mathbb{Z} \\
\hline
\text{CubesOfSquaresOfEvens} = \{ z : \mathbb{Z} \mid z \bmod 2 = 0 \bullet (z * z) * (z * z) * (z * z) \}
\end{array}$$

(c)

$$\begin{array}{|l}
\text{SquareCubePairsOfEvens} : \mathbb{P}(\mathbb{Z} \times \mathbb{Z}) \\
\hline
\text{SquareCubePairsOfEvens} = \{ z_1, z_2 : \mathbb{Z} \mid z_1 \bmod 2 = 0 \wedge z_2 \bmod 2 = 0 \bullet (z_1 * z_1, z_2 * z_2 * z_2) \}
\end{array}$$

(d)

$$\begin{array}{|l}
\text{SquareCubeIdentityPairsOfEvens} : \mathbb{P}(\mathbb{Z} \times \mathbb{Z}) \\
\hline
\text{SquareCubeIdentityPairsOfEvens} = \{ z : \mathbb{Z} \mid z \bmod 2 = 0 \bullet (z * z, z * z * z) \}
\end{array}$$

(e)

$$\begin{array}{|l}
\text{SetsOfSquareOrCubeIntegers} : \mathbb{P}(\mathbb{P} \mathbb{Z}) \\
\hline
\text{SetsOfSquareOrCubeIntegers} = \mathbb{P}\{ z, z_0 : \mathbb{Z} \mid z = z_0 * z_0 \vee z = z_0 * z_0 * z_0 \bullet z \}
\end{array}$$

Question 7

We are designing a prototype for a new podcast platform and player. We are given some initial types and abbreviations. We also define two generic functions `fst()` and `snd()` to provide our fuzz compatible and validated solution capabilities equal to tuple projection, which is not supported in fuzz [?].

$[ShowId, PeopleId]$

$EpisodeId == \mathbb{N}_1$

$Timestamp == \mathbb{N}$

$\frac{[X, Y] \quad fst : X \times Y \rightarrow X}{\forall a : X; b : Y \bullet fst(a, b) = a}$
--

$\frac{[X, Y] \quad snd : X \times Y \rightarrow Y}{\forall a : X; b : Y \bullet snd(a, b) = b}$
--

$\begin{aligned} &podcasts : \mathbb{F} ShowId \\ &podcast_episodes : ShowId \rightarrow \mathbb{F} EpisodeId \\ &episode_info : ShowId \times EpisodeId \rightarrow (\mathbb{F} PeopleId \times Timestamp) \\ &podcast_info : ShowId \rightarrow \mathbb{F} PeopleId \end{aligned}$
$\begin{aligned} &podcasts = \text{dom } podcast_episodes \\ &\forall es : \text{ran } podcast_episodes \bullet \exists n : \mathbb{N} \bullet es = 1 \dots n \\ &\forall s : \text{dom } podcast_episodes \bullet \forall e : podcast_episodes(s) \bullet e \in podcast_episodes(s) \Leftrightarrow (s, e) \in \text{dom } episode_info \\ &podcasts = \text{dom } podcast_info \\ &\forall s : podcasts; e : EpisodeId; p : PeopleId \bullet p \in fst(episode_info(s, e)) \Leftrightarrow p \in podcast_info(s) \\ &\forall s : podcasts; e_1, e_2 : EpisodeId \bullet (((s, e_1) \in \text{dom } episode_info \wedge (s, e_2) \in \text{dom } episode_info) \Rightarrow \\ &\quad ((e_1 < e_2) \Rightarrow (snd(episode_info(s, e_1)) \leq snd(episode_info(s, e_2))))) \end{aligned}$

The above Z is type-checked with fuzz. Each part is briefly explained below:

(a) $podcasts = \text{dom } podcast_episodes$

The set of podcasts is exactly the set of `ShowId` values that have episodes. Expressed as equality rather than mutual subset inclusion.

(b) $\forall es : \text{ran } podcast_episodes \bullet \exists n : \mathbb{N} \bullet es = 1 \dots n$

Each finite set of `EpisodeId` includes an uninterrupted range from 1 to n.

(c)

$\forall s : \text{dom } podcast_episodes \bullet \forall e : podcast_episodes(s) \bullet e \in podcast_episodes(s) \Leftrightarrow (s, e) \in \text{dom } episode_info$

If and only if implies the condition is a bi-implication.

(d) $podcasts \subseteq \text{dom } podcast_info \wedge \text{dom } podcast_info \subseteq podcasts$

If and only if translates to the sets are equal. The same ShowId exist in both sets.

(e)

$\forall s : podcasts; e : EpisodeId; p : PeopleId \bullet p \in fst(episode_info(s, e)) \Leftrightarrow p \in podcast_info(s)$

The specification "exactly when" translates to bi-implication and $fst(episode_info(s, e))$ gives us access to a finite set of PeopleId as does $podcast_info(s)$.

(f)

$\forall s : podcasts; e_1, e_2 : EpisodeId \bullet (((s, e_1) \in \text{dom } episode_info \wedge (s, e_2) \in \text{dom } episode_info) \Rightarrow ((e_1 < e_2) \Rightarrow (snd(episode_info(s, e_1)) \leq snd(episode_info(s, e_2))))))$

We specify how the Timestamp for different episodes of the same show must compare to fulfill the specification by using $snd(episode_info(s, e))$ to access the Timestamp.

Question 8

The following solutions are expressed as abbreviations to enable fuzz validation, which passes for each expression.

(a)

$EightA == \{ s : \text{dom } podcast_episodes \mid \#(podcast_episodes(s)) \geq 100 \}$

Shows with at least 100 episodes. We use the cardinality operator on F EpisodeId after ensuring s is in the domain of $podcast_episodes$.

(b)

$EightB == \{ p : PeopleId \mid (\exists s_1, s_2 : \text{dom } podcast_info \bullet s_1 \neq s_2 \wedge p \in podcast_info(s_1) \wedge p \in podcast_info(s_2)) \}$

The set of PeopleId, which appear in 2 or more shows. We make sure two such shows are distinct.

(c)

$first_episode_timestamp : ShowId \rightarrow Timestamp$	$\forall s : podcasts \bullet (s, 1) \in \text{dom } episode_info \Rightarrow first_episode_timestamp(s) = snd(episode_info(s, 1))$
--	---

$EightC == first_episode_timestamp((\mu s : \text{dom } podcast_info \mid \forall t : \text{dom } podcast_info \bullet s \neq t \wedge \#(podcast_info(s)) > \#(podcast_info(t))))$

We are given that there is exactly one show with the most people and we want the timestamp for the first episode of that show. This involves two operations, finding the show with the most people and finding the timestamp for the first episode of that show. For the first operation, we compare the cardinality of the F PeopleId for every pair s, t : ShowId in the domain of $podcast_info$. This is captured in the mu expression, which returns the single matching value.

For the second operation, we define a partial function `first_episode_timestamp` that yields the Timestamp for `EpisodeId = 1` for $s : ShowId$. We are given there is exactly one episode associated with the most people. Therefore, we know that we will call `first_episode_timestamp` with a `ShowId` that has at least one episode and we are guaranteed that $EpisodeId = 1 \exists$ by constraint 7(b). In addition, calling this function with the s from the μ expression is valid because the constraint defined in 7(e) guarantees that the set of `EpisodeId` in the domain of `podcast_info` and in the first element of the domain of `episode_info` are identical. Therefore, calling this partial function is guaranteed to be valid with the μ expression as its input.

Question 9

We are now modeling an individual podcast player from the point of view of a user. N.B. The free types `PlayedOrNot` and `SavedOrNot` have been adjusted to avoid ambiguity with propositions P and S . The following specification has been type-checked with fuzz.

$PlayedOrNot ::= Played \mid NotP$
 $SavedOrNot ::= Saved \mid NotS$

$my_shows : \mathbb{F} ShowId$ $my_episodes : iseq (ShowId \times EpisodeId)$ $status : ShowId \times EpisodeId \rightarrow (PlayedOrNot \times SavedOrNot)$
$my_shows \subseteq podcasts$ $\forall s : ShowId; e : EpisodeId \bullet (s, e) \in \text{ran } my_episodes \Rightarrow$ $\quad e \in podcast_episodes(s)$ $\forall s : ShowId; e : EpisodeId \bullet (s, e) \in \text{ran } my_episodes \Rightarrow$ $\quad s \in my_shows$ $\forall s : ShowId; e : EpisodeId \bullet (s, e) \in \text{ran } my_episodes \Leftrightarrow (s, e) \in \text{dom } status$

(a) $my_shows \subseteq podcasts$

The elements of the finite set of `ShowId` on this device must be elements of `podcasts`

(b)

$\forall s : ShowId; e : EpisodeId \bullet (s, e) \in \text{ran } my_episodes \Rightarrow$
 $\quad e \in podcast_episodes(s)$

We use the `ran` operator to access the (s, e) in the injective `seq my_episodes` to specify each pair just be a valid mapping in `podcast_episodes`

(c)

$\forall s : ShowId; e : EpisodeId \bullet (s, e) \in \text{ran } my_episodes \Rightarrow$
 $\quad s \in my_shows$

We use the `ran` operator to access the (s, e) in the injective `seq my_episodes` to specify each `ShowId` must be in `my_shows`

(d)

$\forall s : ShowId; e : EpisodeId \bullet (s, e) \in \text{ran } my_episodes \Leftrightarrow (s, e) \in \text{dom } status$

Every (s, e) pair in the `ran my_episodes` has a `status` and every (s, e) in `status` must be in `ran my_episodes`

Question 10

$$\text{Playlist} == \text{seq}((\text{ShowId} \times \text{EpisodeId}) \times \mathbb{N})$$

$$\frac{\text{PossiblePlaylist} : \mathbb{P} \text{ Playlist}}{\text{PossiblePlaylist} = \text{seq}(\text{dom status} \times \mathbb{N})}$$

(a)

$$\frac{\text{length} : \text{PossiblePlaylist} \rightarrow \mathbb{N}}{\text{length}(\langle \rangle) = 0 \wedge (\forall pl : \text{PossiblePlaylist}; ple : (\text{dom status} \times \mathbb{N}) \bullet \text{length}(\langle ple \rangle \frown pl) = \text{snd}(ple) + \text{length}(pl))}$$

(b)

$$\frac{\text{unplayed} : \text{PossiblePlaylist} \rightarrow \text{PossiblePlaylist}}{\text{unplayed}(\langle \rangle) = \langle \rangle \wedge (\forall pl : \text{PossiblePlaylist}; ple : (\text{dom status} \times \mathbb{N}) \bullet (\text{fst}(\text{status}(\text{fst}(ple))) = \text{Played}) \Rightarrow ((\text{unplayed}(\langle ple \rangle \frown pl) = \text{unplayed}(pl)) \wedge (\text{fst}(\text{status}(\text{fst}(ple))) = \text{NotP}) \Rightarrow (\text{unplayed}(\langle ple \rangle \frown pl) = \langle ple \rangle \frown \text{unplayed}(pl))))}$$

(c)

$$\frac{\text{played} : \text{PossiblePlaylist} \rightarrow \text{PossiblePlaylist}}{\text{played}(\langle \rangle) = \langle \rangle \wedge (\forall pl : \text{PossiblePlaylist}; ple : (\text{dom status} \times \mathbb{N}) \bullet (\text{fst}(\text{status}(\text{fst}(ple))) = \text{NotP}) \Rightarrow ((\text{played}(\langle ple \rangle \frown pl) = \text{played}(pl)) \wedge (\text{fst}(\text{status}(\text{fst}(ple))) = \text{Played}) \Rightarrow (\text{played}(\langle ple \rangle \frown pl) = \langle ple \rangle \frown \text{played}(pl))))}$$

Question 11

To use proof by induction, we need to prove the base case and the inductive case. A PossiblePlaylist belongs to $\mathbb{P} \text{ Playlist}$ where Playlist is of type $\text{seq}((\text{ShowId} \text{ cross } \text{EpisodeId}) \text{ cross } \mathbb{N})$. The base case occurs when $p = \langle \rangle$. Therefore, we must first prove $\text{length}(\text{played}(\langle \rangle)) + \text{length}(\text{unplayed}(\langle \rangle)) = \text{length}(\langle \rangle)$

$$\begin{aligned} & \text{length}(\text{played}(\langle \rangle)) + \text{length}(\text{unplayed}(\langle \rangle)) \\ &= \text{length}(\langle \rangle) + \text{length}(\langle \rangle) && [\text{played}(\langle \rangle) = \langle \rangle \wedge \text{unplayed}(\langle \rangle) = \langle \rangle] \\ &= 0 + 0 && [\text{length}(\langle \rangle) = 0] \\ &= 0 && [0 + 0 = 0] \\ &= \text{length}(\langle \rangle) && [\text{length}(\langle \rangle) = 0] \end{aligned}$$

In the above, we use each of the individual $\text{played}()$, $\text{unplayed}()$, and $\text{length}()$ base cases to justify our rewrites.

A playlist element is of type $((\text{dom status}) \text{ cross } \mathbb{N})$. We now seek to prove the inductive hypothesis, which follows the form of induction for sequences and which is an implication [?, page 157].

$$\forall t : \text{PossiblePlaylist}; ple : (\text{dom status} \times \mathbb{N}) \bullet \text{length}(\text{played}(t)) + \text{length}(\text{unplayed}(t)) = \text{length}(t) \Rightarrow \text{length}(\text{played}(\langle ple \rangle \frown t)) + \text{length}(\text{unplayed}(\langle ple \rangle \frown t)) = \text{length}(\langle ple \rangle \frown t)$$

The status of a playlist element is of type $(\text{PlayedOrNot}, \text{SavedOrNot})$. The inductive hypothesis proof begins by establishing that there are two mutually exclusive and exhaustive *cases* : *one* where the playlist element's PlayedOrNot free type value is Played and another where it is NotP . The proof will establish that for both cases, the same conclusion can be proven. The proof relies upon the inductive hypothesis $\text{length}(\text{played}(t)) + \text{length}(\text{unplayed}(t)) = \text{length}(t)$.

Case 1: The playlist element is played, which is represented by $\text{fst}(\text{status} \text{ fst}(ple)) = \text{Played}$, which says take the status of the ple and inspect the first element of the status tuple and determine whether $\text{PlayedOrNot} = \text{Played}$.

$$\begin{aligned}
& \text{length}(\text{played}(\langle \text{ple} \rangle \hat{\ } t)) + \text{length}(\text{unplayed}(\langle \text{ple} \rangle \hat{\ } t)) \\
&= \text{length}(\langle \text{ple} \rangle \hat{\ } \text{played}(t)) + \text{length}(\text{unplayed}(\langle \text{ple} \rangle \hat{\ } t)) & [\text{played}(\langle \text{ple} \rangle \hat{\ } \text{pl}) = \langle \text{ple} \rangle \hat{\ } (\text{played}(\text{pl}))] \\
&= \text{length}(\langle \text{ple} \rangle \hat{\ } \text{played}(t)) + \text{length}(\text{unplayed}(t)) & [\text{unplayed}(\langle \text{ple} \rangle \hat{\ } \text{pl}) = \text{unplayed}(\text{pl})] \\
&= \text{snd}(\text{ple}) + \text{length}(\text{played}(t)) + \text{length}(\text{unplayed}(t)) & [\text{length}(\langle \text{ple} \rangle \hat{\ } \text{pl}) = \text{snd}(\text{ple}) + \text{length}(\text{pl})] \\
&= \text{snd}(\text{ple}) + (\text{length}(\text{played}(t)) + \text{length}(\text{unplayed}(t))) & [\text{associativity of addition}] \\
&= \text{snd}(\text{ple}) + \text{length}(t) & [\text{length}(\text{played}(p)) + \text{length}(\text{unplayed}(p)) = \text{length}(p)] \\
&= \text{length}(\langle \text{ple} \rangle \hat{\ } t) & [\text{length}(\langle \text{ple} \rangle \hat{\ } \text{pl}) = \text{snd}(\text{ple}) + \text{length}(\text{pl})]
\end{aligned}$$

We first rewrite the played term as we know ple is Played and then we simplify the unplayed term as we know ple is not NotP. From here, we apply the definition of length for a single sequence element concatenated to a sequence. Using the associative property of addition, we arrive at a form of the inductive hypothesis that we can simplify. From there, we apply our definition of length again to arrive at our final form.

Case 2: The playlist element is not played, which is represented by $\text{fst}(\text{status } \text{fst}(\text{ple})) = \text{NotP}$, which says take the status of the ple and inspect the first element of the status tuple and determine whether $\text{PlayedOrNot} = \text{NotP}$.

$$\begin{aligned}
& \text{length}(\text{played}(\langle \text{ple} \rangle \hat{\ } t)) + \text{length}(\text{unplayed}(\langle \text{ple} \rangle \hat{\ } t)) \\
&= \text{length}(\text{played}(\langle \text{ple} \rangle \hat{\ } t)) + \text{length}(\langle \text{ple} \rangle \hat{\ } \text{unplayed}(t)) & [\text{unplayed}(\langle \text{ple} \rangle \hat{\ } \text{pl}) = \langle \text{ple} \rangle \hat{\ } (\text{unplayed}(\text{pl}))] \\
&= \text{length}(\text{played}(t)) + \text{length}(\langle \text{ple} \rangle \hat{\ } \text{unplayed}(t)) & [\text{played}(\langle \text{ple} \rangle \hat{\ } \text{pl}) = \text{played}(\text{pl})] \\
&= \text{length}(\text{played}(t)) + \text{snd}(\text{ple}) + \text{length}(\text{unplayed}(t)) & [\text{length}(\langle \text{ple} \rangle \hat{\ } \text{pl}) = \text{snd}(\text{ple}) + \text{length}(\text{pl})] \\
&= \text{snd}(\text{ple}) + \text{length}(\text{played}(t)) + \text{length}(\text{unplayed}(t)) & [\text{commutativity of addition}] \\
&= \text{snd}(\text{ple}) + (\text{length}(\text{played}(t)) + \text{length}(\text{unplayed}(t))) & [\text{associativity of addition}] \\
&= \text{snd}(\text{ple}) + \text{length}(t) & [\text{length}(\text{played}(p)) + \text{length}(\text{unplayed}(p)) = \text{length}(p)] \\
&= \text{length}(\langle \text{ple} \rangle \hat{\ } t) & [\text{length}(\langle \text{ple} \rangle \hat{\ } \text{pl}) = \text{snd}(\text{ple}) + \text{length}(\text{pl})]
\end{aligned}$$

We first rewrite the unplayed term as we know ple is NotP and then we simplify the played term as we know ple is not Played. From here, we apply the definition of length for a single sequence element concatenated to a sequence. We must re-arrange our addends using the commutative property of addition. Next, we use the associative property of addition to arrive at a form of the inductive hypothesis that we can simplify. From there, we apply our definition of length again to arrive at our final form.

Since the base case is proven and both inductive cases result in the same outcome, we can use the implication introduction rule $[\Rightarrow \text{intro}]$ to unify our cases and conclude:

$$\frac{\forall p : \text{PossiblePlaylist} \bullet \text{length}(\text{played}(p)) + \text{length}(\text{unplayed}(p))}{\forall p : \text{PossiblePlaylist} \bullet \text{length}(p)}$$

Question 12

First, we can create an abbreviation for the elements of our list to make our expressions less cumbersome.

$$\text{ListElement} == (\text{ShowId} \times \text{EpisodeId}) \times (\mathbb{N} \times \text{PlayedOrNot} \times \text{SavedOrNot})$$

We restate our List free type definition using the abbreviation as

$$\text{List} ::= \text{nil} \mid \text{join} \langle \langle \text{ListElement} \times \text{List} \rangle \rangle$$

For example, such a List could include $(\text{join}((1, 2), (3600, \text{NotP}, \text{Saved}), (\text{join}((1, 1), (3600, \text{Played}, \text{Saved})), \text{nil})))$ which is of the form $(\text{join}(e, \text{join}(e, \text{nil})))$.

(a)

We define the induction principle for this List for the proposition Q, building upon the List of natural numbers example found in section 13.3 of the course notes [?, page 162].

$$\frac{Q(\text{nil}) \quad \forall e : \text{ListElement}; l : \text{List} \bullet Q(l) \Rightarrow Q(\text{join}(e, l))}{\forall l : \text{List} \bullet Q(l)}$$

(b)

Fuzz does not support Z tuple projection such as e.2. Therefore, we define generic access functions for each element of any triple, which we will use with ListElement, together with the previously specified fst() and snd() for tuples.

$$\frac{[A, B, C] \text{-----}}{\text{fstOfTriple} : A \times B \times C \rightarrow A}$$

$$\forall a : A; b : B; c : C \bullet \text{fstOfTriple}(a, b, c) = a$$

$$\frac{[A, B, C] \text{-----}}{\text{sndOfTriple} : A \times B \times C \rightarrow B}$$

$$\forall a : A; b : B; c : C \bullet \text{sndOfTriple}(a, b, c) = b$$

$$\frac{[A, B, C] \text{-----}}{\text{thrdOfTriple} : A \times B \times C \rightarrow C}$$

$$\forall a : A; b : B; c : C \bullet \text{thrdOfTriple}(a, b, c) = c$$

Next, we specify our recursive functions

$$\frac{\text{length_L} : \text{List} \rightarrow \mathbb{N}}{\text{length_L}(\text{nil}) = 0}$$

$$\forall e : \text{ListElement}; l : \text{List} \bullet \text{length_L}(\text{join}(e, l)) = \text{fstOfTriple}(\text{snd}(e)) + \text{length_L}(l)$$

The length of a List is the recursive sum of *seconds* : N, found in the first position of the triple found in the second element of the tuple of each ListElement.

$$\frac{\text{unplayed_L} : \text{List} \rightarrow \text{List}}{\text{unplayed_L}(\text{nil}) = \text{nil}}$$

$$\forall e : \text{ListElement}; l : \text{List} \bullet \text{sndOfTriple}(\text{snd}(e)) = \text{NotP} \Rightarrow \text{unplayed_L}(\text{join}(e, l)) = \text{join}(e, \text{unplayed_L}(l))$$

$$\forall e : \text{ListElement}; l : \text{List} \bullet \text{sndOfTriple}(\text{snd}(e)) = \text{Played} \Rightarrow \text{unplayed_L}(\text{join}(e, l)) = \text{unplayed_L}(l)$$

The unplayed list of any given list is a new list, which recursively includes or excludes each ListElement based on the PlayedOrNot free type value of NotP found in the second position of the triple found in the second element of the tuple of each ListElement.

$$\frac{\text{played_L} : \text{List} \rightarrow \text{List}}{\text{played_L}(\text{nil}) = \text{nil}}$$

$$\forall e : \text{ListElement}; l : \text{List} \bullet \text{sndOfTriple}(\text{snd}(e)) = \text{Played} \Rightarrow \text{played_L}(\text{join}(e, l)) = \text{join}(e, \text{played_L}(l))$$

$$\forall e : \text{ListElement}; l : \text{List} \bullet \text{sndOfTriple}(\text{snd}(e)) = \text{NotP} \Rightarrow \text{played_L}(\text{join}(e, l)) = \text{played_L}(l)$$

The played list of any given list is a new list, which recursively includes or excludes each ListElement based on the PlayedOrNot free type value of Played found in the second position of the triple found in the second element of the tuple of each ListElement.

(c)

To use proof by induction, we need to prove the base case and the inductive case.

$$\forall l : List \bullet length_L(played_L(l)) + length_L(unplayed_L(l)) = length_L(l)$$

The base case occurs when the List is nil.

$$\begin{aligned} & length_L(played_L(nil)) + length_L(unplayed_L(nil)) \\ &= length_L(nil) + length_L(nil) && [(un)played_L(nil)=nil] \\ &= 0 + 0 && [length_L(nil)=0] \\ &= 0 && [arithmetic] \\ &= length_L(nil) && [length_L(nil)=0] \end{aligned}$$

In the above, we use each of the definitions of played_L(nil), unplayed_L(nil), and length_L(nil) to prove our base case. Now we have to move to our inductive case

$$\begin{aligned} & length_L(played(l)) + length_L(unplayed(l)) = length_L(l) \Rightarrow \\ & length_L(played(join(e, l))) + length_L(unplayed(join(e, l))) = length_L(join(e, l)) \end{aligned}$$

To establish this implication is true, we must prove it all cases. For this statement, $e : ListElement$ is included in played_L's output or unplayed_L's output based on the value of its free type PlayedOrNot, which can only have one of two states that map directly to these two outcomes. Therefore, we have two cases, one where e's PlayedOrNot value is Played and another where the value is NotP.

Case 1: e 's $PlayedOrNot = Played$

$$\begin{aligned}
& length_L(played(join(e, l))) + length_L(unplayed(join(e, l))) \\
&= length_L(played(join(e, l))) + length_L(unplayed(l)) && [\text{def of } unplayed_L(join(e, l))] \\
&= length_L(join(e, played(l))) + length_L(unplayed(l)) && [\text{def of } played_L(join(e, l))] \\
&= fstOfTriple(snd(e)) + length_L(played(l)) + length_L(unplayed(l)) && [\text{def of } length_L(join(e, l))] \\
&= fstOfTriple(snd(e)) + (length_L(played(l)) + length_L(unplayed(l))) && [\text{associativity of addition}] \\
&= fstOfTriple(snd(e)) + length_L(l) && [\text{inductive hypothesis}] \\
&= length_L(join(e, l)) && [\text{def of } length_L(join(e, l))]
\end{aligned}$$

Case 2: e 's $PlayedOrNot = NotP$

$$\begin{aligned}
& length_L(played(join(e, l))) + length_L(unplayed(join(e, l))) \\
&= length_L(played(l)) + length_L(unplayed(join(e, l))) && [\text{def of } played_L(join(e, l))] \\
&= length_L(played(l)) + length_L(join(e, unplayed(l))) && [\text{def of } unplayed_L(join(e, l))] \\
&= length_L(played(l)) + fstOfTriple(snd(e)) + length_L(unplayed(l)) && [\text{def of } length_L(join(e, l))] \\
&= fstOfTriple(snd(e)) + length_L(played(l)) + length_L(unplayed(l)) && [\text{commutativity of addition}] \\
&= fstOfTriple(snd(e)) + (length_L(played(l)) + length_L(unplayed(l))) && [\text{associativity of addition}] \\
&= fstOfTriple(snd(e)) + length_L(l) && [\text{inductive hypothesis}] \\
&= length_L(join(e, l)) && [\text{def of } length_L(join(e, l))]
\end{aligned}$$

Having shown both cases result in the same outcome, we can unify our cases and conclude:

$$\frac{\forall l : List \bullet length_L(played(l)) + length_L(unplayed(l)) = length_L(l)}{\forall l : List \bullet length_L(played(join(e, l))) + length_L(unplayed(join(e, l))) = length_L(join(e, l))}$$

Since the base case is proven and both inductive cases result in the same outcome, we can use the implication introduction rule $[\Rightarrow \text{intro}]$ to unify our cases (n.b. $length_L$ is abbreviated L below to spare the reader further font scaling) and conclude:

$$\frac{L(nil) \quad \forall l : List \bullet L(played(l)) + L(unplayed(l)) = L(l) \Rightarrow L(played(join(e, l))) + L(unplayed(join(e, l))) = L(join(e, l))}{\forall l : List \bullet L(l)}$$

we have proven $\forall l : List \bullet length_L(played_L(l)) + length_L(unplayed_L(l)) = length_L(l)$